

Вступ до функціонального програмування. Методичні вказівки з виконання лабораторних робіт

Лабораторна робота 1. Обробка списків з використанням базових функцій

Мета лабораторної роботи: ознайомитись із базовими типами даних та функціями Common Lisp, отримати практичні навички роботи зі списками.

Опис базових типів даних, базових функцій, а також особливостей роботи з REPL та внутрішньої організації списків наведено в розділах 1.3, 2.1, 2.3.1 навчального посібника.

Реченець виконання роботи (ака **дедлайн**) визначається викладачем під час видачі завдання на лабораторну роботу.

1.1 Завдання

Завдання складається з двох частин: загальної для всіх та завдання за варіантом.

1.1.1 Загальне завдання

Загальне завдання складається з кількох пунктів, наведених нижче. Для зручності перевірки результатів, у звіті перед фрагментом лістингу виконання кожного з пунктів загального завдання варто додати коментар з позначенням пункту. Наприклад:

```
;; Пункт 1
CL-USER> ...

;; Пункт 2
CL-USER> ...
```

Або ж розділити лістинг по пунктам і додати підзаголовки (див. п. 1.2 для детальнішого пояснення).

Завдання:

1. Створіть список з п'яти елементів, використовуючи функції `LIST` і `CONS`. Форма створення списку має бути одна — використання `SET` чи `SETQ` (або інших допоміжних форм) для збереження проміжних значень не допускається. Загальна кількість елементів (включно з підсписками та їх елементами) не має перевищувати 10-12 шт. (дуже великий список робити не потрібно). Збережіть створений список у якусь змінну з `SET` або `SETQ`. Список має містити (напрямую або у підсписах):
 - хоча б один символ
 - хоча б одне число

- хоча б один не пустий підсписок

- хоча б один пустий підсписок

2. Отримайте голову списку.

3. Отримайте хвіст списку.

4. Отримайте третій елемент списку.

5. Отримайте останній елемент списку.

6. Використайте предикати `ATOM` та `LISTP` на різних елементах списку (по 2-3 приклади для кожної функції).

7. Використайте на елементах списку 2-3 інших предикати з розглянутих у розділі 4 навчального посібника.

8. Об'єднайте створений список з одним із його непустих підсписків. Для цього використайте функцію `APPEND`.

1.1.2 Завдання за варіантом

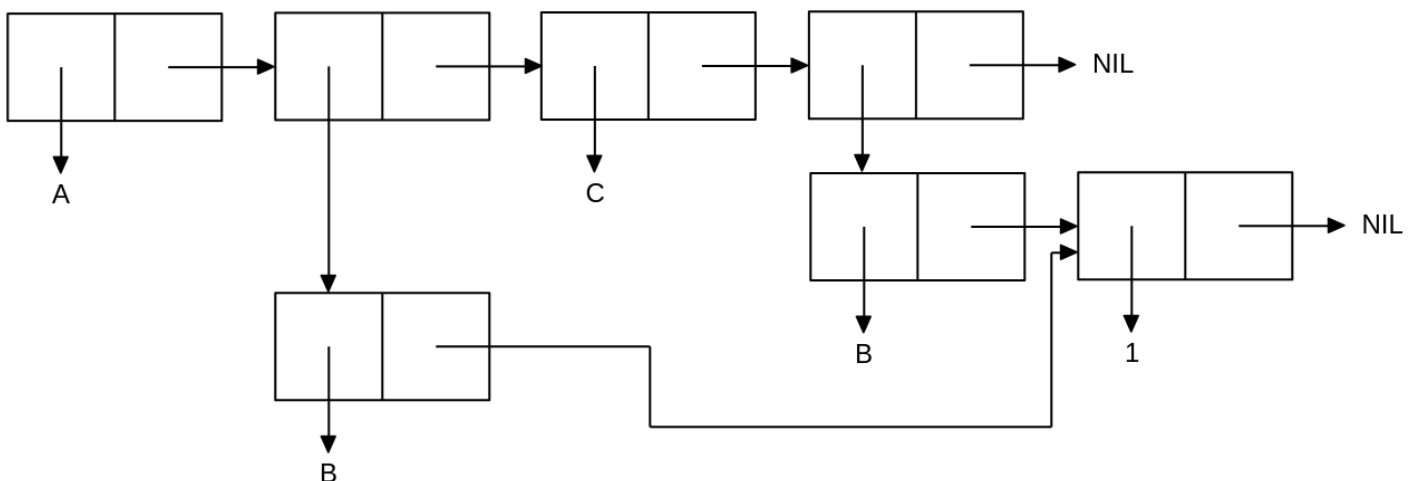
Створіть список, що відповідає структурі списку, наведеній на рисунку (за варіантом).

Для цього **допускається використання не більше двох форм конструювання списку на "верхньому рівні"**. Але аргументами цих форм можуть бути результати інших викликів форм конструювання списків. Наприклад:

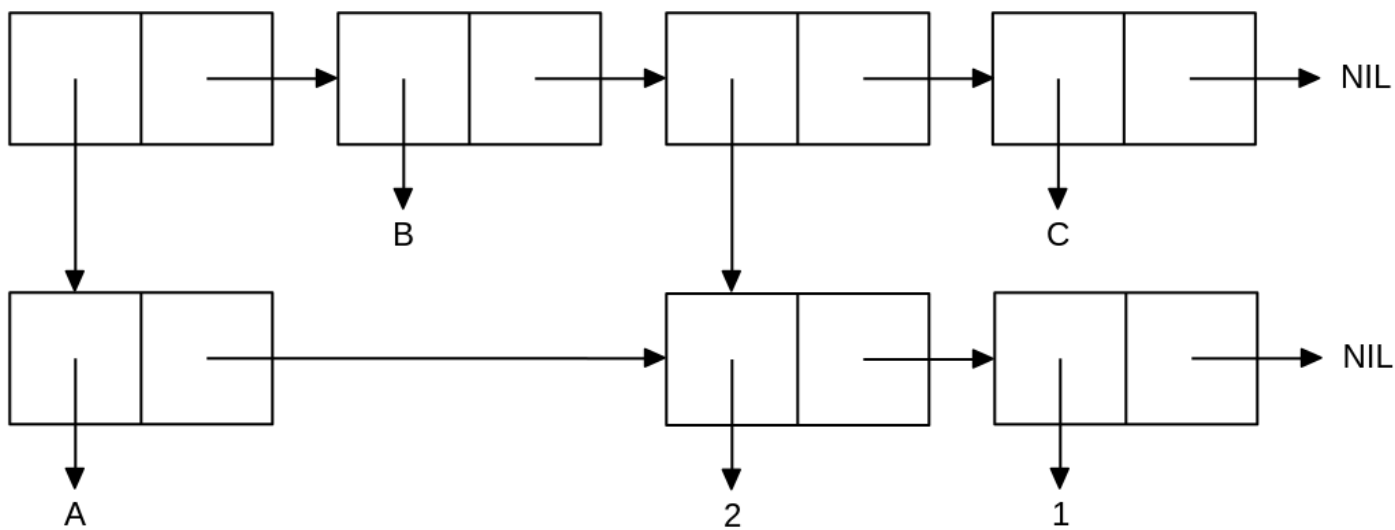
```
(let ((x (list 1 2))) ; (list ...) - перша форма
    ;; далі - друга форма (list ...), що містить підсписки
    (list x (list 3) 4 (cons (cdr x) '(6)))))
```

Номер варіанту обирається як номер у списку групи, який надсилає викладач на початку семестру (на випадок, якщо протягом семестру стануться зміни в складі групи), за модулем 8: 1 -> 1, 2 -> 2, ..., 9 -> 1, 10 -> 2, ...

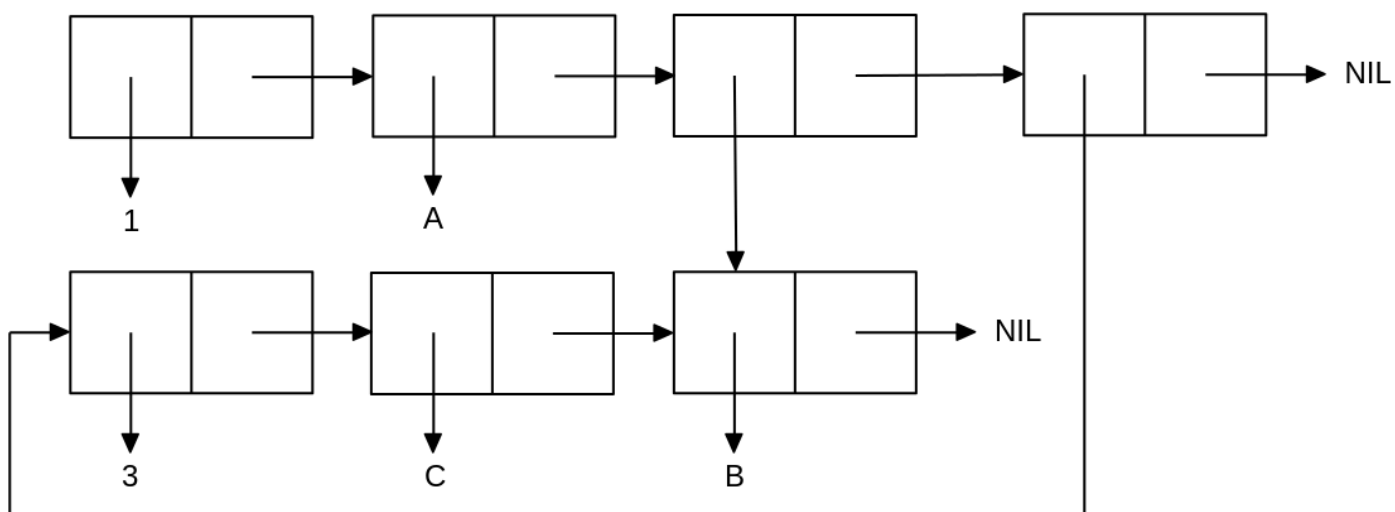
Варіант 1



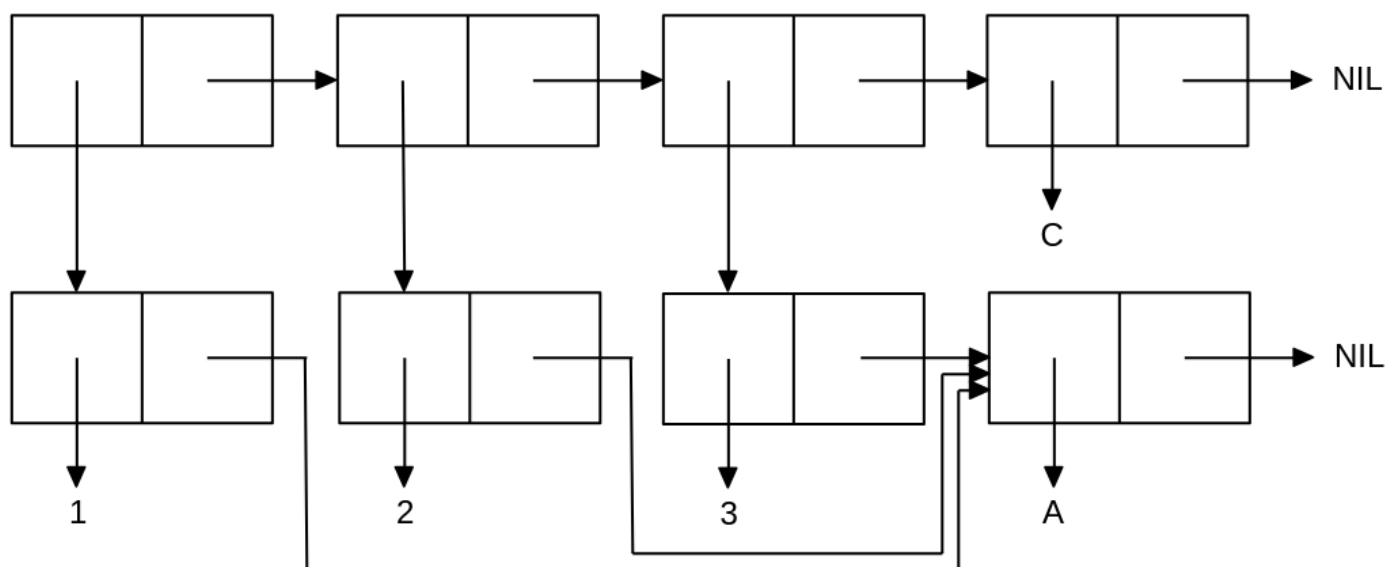
Варіант 2



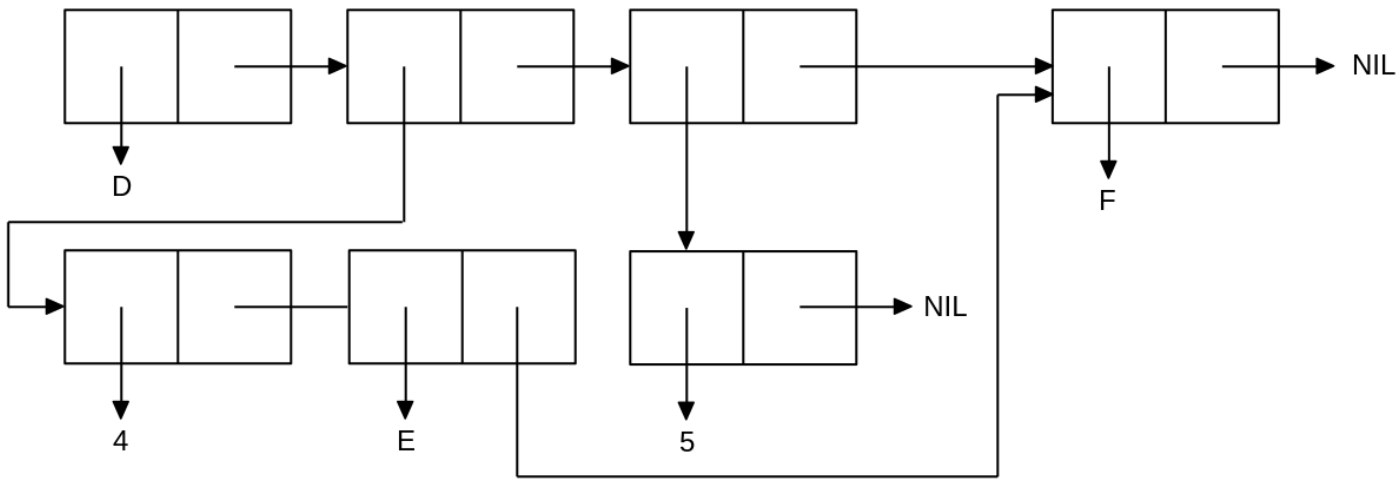
Варіант 3



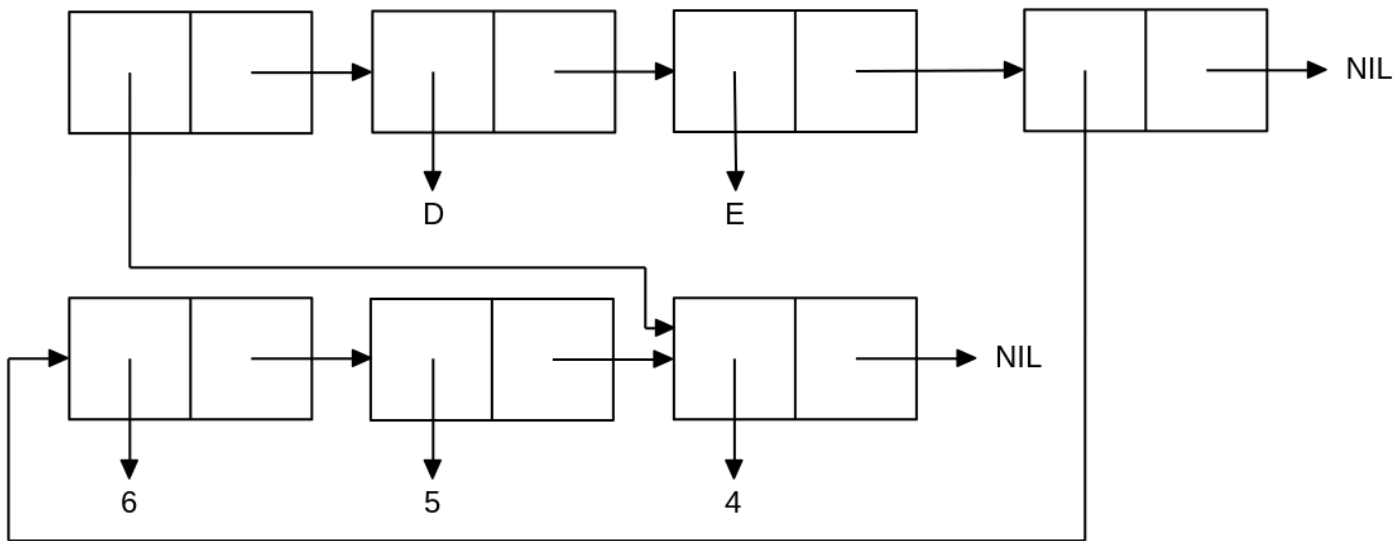
Варіант 4



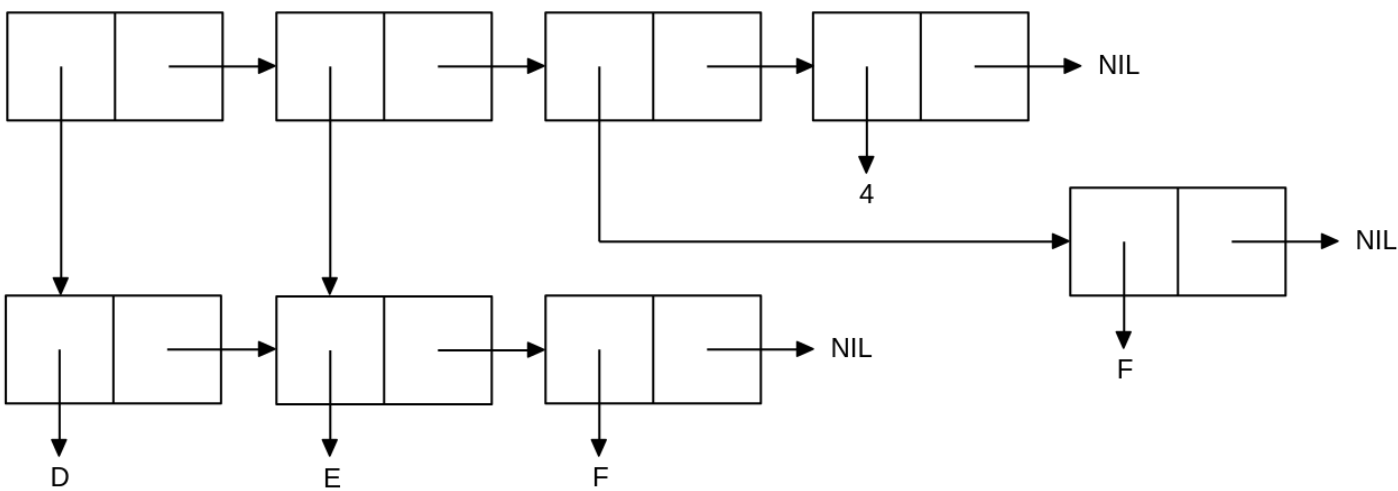
Варіант 5



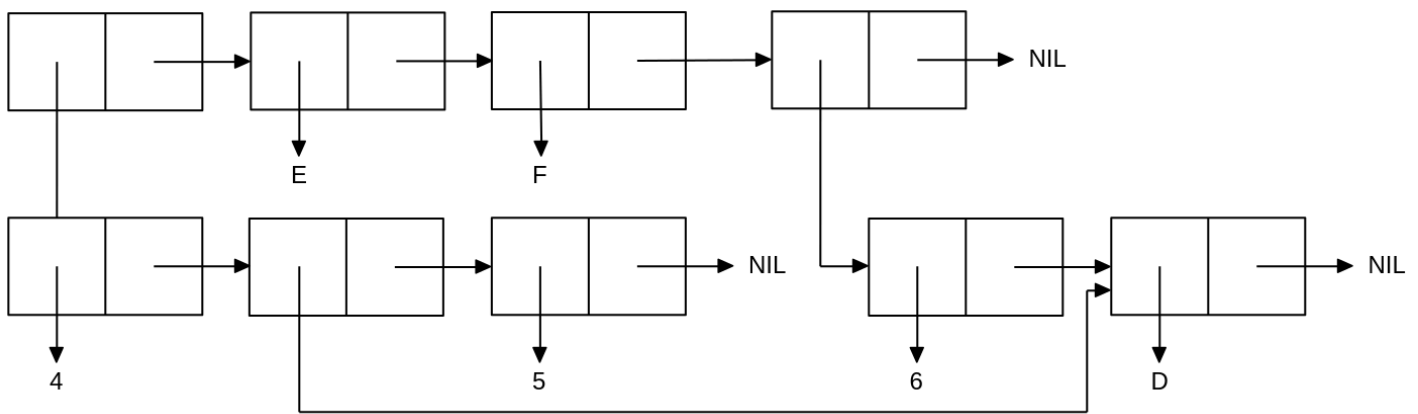
Варіант 6



Варіант 7



Варіант 8



1.2 Формат звіту та захисту

В результаті виконання лабораторної роботи необхідно підготувати звіт у форматі Markdown. Файл звіту (з розширенням `.md`) та файл із зображенням структури списку за варіантом (що використовуватиметься у звіті) або посилання на файл звіту у репозиторії (GitHub, Bitbucket, тощо.) необхідно надіслати викладачу на електронну пошту.

Звіт має містити титульні заголовки (див. шаблон), лістинг викликів в REPL з виконанням загального завдання, номер варіанту та лістинг виконання завдання за варіантом.

Шаблон звіту такий:

```
<p align="center"><b>МОНУ НТУУ КПІ ім. Ігоря Сікорського ФПМ СПІСКС</b></p>
```

```
<p align="center">
```

```
<b>Звіт з лабораторної роботи 1</b><br/>
```

```
"Обробка списків з використанням базових функцій"<br/>
```

```
дисципліни "Вступ до функціонального програмування"
```

```
</p>
```

```
<p align="right"><b>Студент(-ка)</b>: Прізвище Ім'я По-батькові група</p>
```

```
<p align="right"><b>Рік</b>: рік</p>
```

Загальне завдання

```
<!--
```

```
лістинг пунктів загального завдання можна навести або в одному блоці коду
із коментарями, що позначають початок виконання окремих пунктів, або ж
розділити весь лістинг на окремі блоки коду і додати для них підзаголовки
(напр. ### Пункт 1).
```

```
-->
```

```
```lisp
```

```
;; Тут має бути лістинг (текст) виконання загального завдання. Оскільки
;; завдання вимагає лише виконання простих команд, достатньо буде додати
;; лістинг у вигляді тексту виконання команд і отриманого результату з REPL
;; Наприклад:
```

```
;; Пункт 1
```

```
CL-USER> (setq my-list ...)
(...)
```

```
;; Пункт 2
```

```
CL-USER> ...
```

```
:::
```

```
Варіант <номер варіанту>
```

```
<p align="center">
```

```

```

```
</p>
```

```
```lisp
```

```
;; тут має бути лістинг (текст) виконання завдання за варіантом - також бажано
```

```
;; у вигляді пословності виконання команд в REPL і отриманого результату
```

```
```
```

*Примітка:* за потреби, опис правил форматування Markdown можна знайти за посиланнями:

- <https://markdown.rozh2sch.org.ua/>
- <https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

На захисті лабораторної роботи необхідно продемонструвати роботу окремих форм, відповісти на питання викладача по виконаній роботі та теорії. Контрольні питання наведені в п 1.3.

## 1.3 Контрольні питання

1. Схарактеризуйте базові типи Common Lisp.
2. Що таке REPL?
3. Які є базові функції для роботи зі списками?
4. Які є базові функції для виконання арифметичних операцій?
5. Які є предикати для порівняння об'єктів в Common Lisp?
6. Як в Common Lisp реалізовані логічні операції?
7. Що таке список, спискова комірка, точкова пара?
8. В чому різниця між функцією, макросом та спеціальним оператором (формою)?
9. Наведіть приклади спеціальних операторів.
10. Як можна відмінити обчислення або ж "виконати" форму?

## Лабораторна робота 2. Рекурсія

**Мета лабораторної роботи:** навчитись застосовувати рекурсію при реалізації простих програм мовою Common Lisp.

Різні види рекурсії розглядаються в розділі 3 навчального посібника. Визначення функції

та форми для керування потоком виконання розглядаються в розділах 2.4.1 і 2.6 навчального посібника. Зв'язування змінних та контекст дії зв'язування розглядається в розділі 2.5 посібника.

**Реченець** виконання роботи (ака **дедлайн**) визначається викладачем під час видачі завдання на лабораторну роботу.

## 2.1 Завдання

Реалізуйте дві рекурсивні функції, що виконують деякі дії з вхідним(и) списком(-ами), за можливості/необхідності використовуючи різні види рекурсії. Функції, які необхідно реалізувати, задаються варіантом (п. 2.1.1). Вимоги до функцій:

1. Зміна списку згідно із завданням має відбуватись за рахунок конструювання нового списку, а не зміни наявного (вхідного).
2. Не допускається використання функцій вищого порядку чи стандартних функцій для роботи зі списками, що не наведені в четвертому розділі навчального посібника.
3. Реалізована функція не має бути функцією вищого порядку, тобто приймати функції в якості аргументів.
4. Не допускається використання псевдофункцій (деструктивного підходу).
5. Не допускається використання циклів.

Кожна реалізована функція має бути протестована для різних тестових наборів. Тести мають бути оформленні у вигляді модульних тестів (див. п. 2.3).

*Додатковий бал* за лабораторну роботу можна отримати в разі виконання всіх наступних умов:

- робота виконана до дедлайну (включно з датою дедлайну)
- крім основних реалізацій функцій за варіантом, також реалізовано додатковий варіант однієї чи обох функцій, який працюватиме швидше за основну реалізацію, не порушуючи при цьому перші три вимоги до основної реалізації (вимоги 4 і 5 можуть бути порушені), за виключенням того, що в разі необхідності можна також використати стандартну функцію `copy-list`

### 2.1.1 Варіанти завдань

Номер варіанту обирається як номер у списку групи, який надсилає викладач на початку семестру, *за модулем 15*: 1 -> 1, 2 -> 2, ..., 16 -> 1, 17 -> 2, ...

#### Варіант 1

1. Написати функцію `reverse-and-nest-head`, яка обертає вхідний список та утворює вкладену структуру з підсписків з його елементами, починаючи з голови:

```
CL-USER> (reverse-and-nest-head '(a b c))
(((C) B) A)
```

- Написати функцію `duplicate-elements` , що дублює елементи вхідного списку задану кількість разів:

```
CL-USER> (duplicate-elements '(a b c) 3)
(A A A B B B C C C)
```

## Варіант 2

- Написати функцію `remove-seconds-and-thirds` , яка видаляє зі списку кожен другий і третій елементи:

```
CL-USER> (remove-seconds-and-thirds '(a b c d e f g))
(A D G)
```

- Написати функцію `list-set-intersection` , яка визначає перетин двох множин, заданих списками атомів:

```
CL-USER> (list-set-intersection '(1 2 3 4) '(3 4 5 6))
(3 4) ; порядок може відрізнятись
```

## Варіант 3

- Написати функцію `group-pairs` , яка групує послідовні пари елементів у списки:

```
CL-USER> (group-pairs '(a b c d e f g))
((A B) (C D) (E F) (G))
```

- Написати функцію `list-set-union` , яка визначає об'єднання двох множин, заданих списками атомів:

```
CL-USER> (list-set-union '(1 2 3) '(2 3 4))
(1 2 3 4) ; порядок може відрізнятись
```

## Варіант 4

- Написати функцію `remove-seconds-and-reverse` , яка видаляє зі списку кожен другий елемент і обертає результат у зворотному порядку:

```
CL-USER> (remove-seconds-and-reverse '(1 2 a b 3 4 d))
(D 3 A 1)
```

- Написати функцію `list-set-difference` , яка визначає різницю двох множин, заданих списками атомів:

```
CL-USER> (list-set-difference '(1 2 3 4) '(3 4 5 6))
(1 2) ; порядок може відрізнятись
```

## Варіант 5

- Написати функцію `remove-seconds` , яка видаляє зі списку кожен другий елемент:

```
CL-USER> (remove-seconds '(1 2 a b 3 4 d))
(1 A 3 D)
```

2. Написати функцію `list-set-symmetric-difference`, яка визначає симетричну різницю двох множин, заданих списками атомів (тобто, множину елементів, що не входять до обох множин):

```
CL-USER> (list-set-symmetric-difference '(1 2 3 4) '(3 4 5 6))
(1 2 5 6) ; порядок може відрізнятись
```

## Варіант 6

1. Написати функцію `merge-lists-spinning-pairs`, яка групує відповідні елементи двох списків, по чергово змінюючи їх взаємне розташування в групі:

```
CL-USER> (merge-lists-spinning-pairs '(1 2 3 4 5) '(a b c d))
((1 A) (B 2) (3 C) (D 4) (5))
```

2. Написати предикат `list-set-intersect-p`, який визначає чи перетинаються дві множини, задані списками атомів, чи ні:

```
CL-USER> (list-set-intersect-p '(1 2 3) '(4 5 6))
NIL
CL-USER> (list-set-intersect-p '(1 2 3) '(3 4 5))
T
```

## Варіант 7

1. Написати функцію `merge-lists-pairs`, яка групує відповідні елементи двох списків:

```
CL-USER> (merge-lists-pairs '(1 2 3 4 5) '(a b c d))
((1 A) (2 B) (3 C) (4 D) (5))
```

2. Написати предикат `sublist-after-p`, який перевіряє, чи знаходиться після визначеного елемента списку атомів визначена послідовність елементів (список):

```
CL-USER> (sublist-after-p '(1 a 2 b 3 c 4 d) 'b '(3 c))
T
CL-USER> (sublist-after-p '(1 a 2 b 3 c 4 d) 'b '(3 c d))
NIL
```

## Варіант 8

1. Написати функцію `reverse-and-nest-tail`, яка обертає вхідний список та утворює вкладену структуру з підсписків з його елементами, починаючи з хвоста:

```
CL-USER> (reverse-and-nest-tail '(a b c))
(C (B (A)))
```

2. Написати функцію `compress-list`, яка заміщає сукупності послідовно розташованих однакових елементів списку, що складається з атомів,

двоелементними списками виду (<кількість-повторень> <елемент>) :

```
CL-USER> (compress-list '(1 a a 3 3 3 b))
((1 1) (2 A) (3 3) (1 B))
```

## Варіант 9

1. Написати функцію `remove-even-pairs` , яка видаляє зі списку кожен третій та четвертий елементи:

```
CL-USER> (remove-even-pairs '(1 a 2 b 3 c 4))
(1 A 3 C)
```

2. Написати функцію `sublist` , яка поверне підсписок списку-аргументу у визначеному проміжку:

```
CL-USER> (sublist '(1 2 3 4 5) 0 1)
(1)
CL-USER> (sublist '(1 2 3 4 5) 1 3)
(2 3)
```

## Варіант 10

1. Написати функцію `group-triples` , яка групує послідовні трійки елементів у списки:

```
CL-USER> (group-triples '(a b c d e f g))
((A B C) (D E F) (G))
```

2. Написати функцію `list-set-intersection-3` , яка визначає перетин трьох множин, заданих списками атомів:

```
CL-USER> (list-set-intersection-3 '(1 2 3 4) '(3 4 5 6) '(1 3 4 6))
(3 4) ; порядок може відрізнатись
```

## Варіант 11

1. Написати функцію `remove-thirds` , яка видаляє зі списку кожен третій елемент:

```
CL-USER> (remove-thirds '(a b c d e f g))
(A B D E G)
```

2. Написати функцію `list-set-union-3` , яка визначає об'єднання трьох множин, заданих списками атомів:

```
CL-USER> (list-set-union-3 '(1 2 3) '(2 3 4) '(nil t))
(1 2 3 4 NIL T) ; порядок може відрізнатись
```

## Варіант 12

1. Написати функцію `remove-thirds-and-reverse` , яка видаляє зі списку кожен третій елемент і обертає результат у зворотному порядку:

```
CL-USER> (remove-thirds-and-reverse '(a b c d e f g))
(G E D B A)
```

2. Написати функцію `list-set-difference-3`, яка визначає різницю трьох множин, заданих списками атомів:

```
CL-USER> (list-set-difference '(1 2 3 4) '(4 5 6) '(2 5 7))
(1 3) ; порядок може відрізнятись
```

### Варіант 13

1. Написати функцію `remove-even-triples`, яка видалить парні трійки послідовних елементів зі списку:

```
CL-USER> (remove-even-triples '(a b c d e f g h))
(A B C G H)
```

2. Написати функцію `decompress-list`, яка "розпакує" зі списку пар виду (кількість-повторень елемент) послідовність елементів визначеної кількості:

```
CL-USER> (decompress-list '((1 1) (2 a) (3 3) (1 4)))
(1 A A 3 3 3 4)
```

### Варіант 14

1. Написати функцію `group-pairs-and-reverse`, яка групує послідовні пари елементів у списки і повертає обернений результат:

```
CL-USER> (group-pairs-and-reverse '(a b c d e f g))
((G) (E F) (C D) (A B))
```

2. Написати функцію `cyclic-shift-left`, яка виконує циклічний зсув елементів списку ліворуч на задане число позицій:

```
CL-USER> (cyclic-shift-left '(1 2 3 4 5 6) 2)
(3 4 5 6 1 2)
```

### Варіант 15

1. Написати функцію `spread-values`, яка замінює `nil` в списку на попередній `nil` елемент:

```
CL-USER> (spread-values '(nil 1 2 nil 3 nil nil 4 5))
(NIL 1 2 2 3 3 3 4 5)
```

2. Написати функцію `delete-duplicates-sequence`, яка видаляє всі послідовні дублікати тих елементів з вхідного списку атомів, послідовних дублікатів яких більше за задане число:

```
CL-USER> (delete-duplicates-sequence '(1 1 2 3 3 3 2 2 a a a b) 3)
(1 1 2 3 2 2 A B)
```

## 2.2 Формат звіту та захисту

В результаті виконання лабораторної роботи необхідно підготувати звіт у форматі Markdown. Файл звіту (з розширенням `.md`) та файл(и) з кодом (з розширенням `.lisp` або `.lisp`), або посилання на файл звіту і файл(и) з кодом у репозиторії (GitHub, Bitbucket, тощо.) необхідно надіслати викладачу на електронну пошту.

Звіт має містити (див. шаблон):

- титульні заголовки
- загальне завдання
- номер варіанту
- завдання за варіантом
- лістинг реалізованих завдань
- лістинг реалізації тестових наборів та тестових утиліт
- результат виконання тестових наборів

Шаблон звіту такий:

```
<p align="center">МОНУ НТУУ КПІ ім. Ігоря Сікорського ФПМ СПІСКС</p>

<p align="center">
Звіт з лабораторної роботи 2

"Рекурсія"

дисципліни "Вступ до функціонального програмування"
</p>

<p align="right">Студент(-ка): Прізвище Ім'я По-батькові група</p>
<p align="right">Рік: рік</p>

Загальне завдання

<!-- Зазначається загальне завдання -->

Варіант <номер варіанту>

<!-- Зазначається завдання за варіантом -->

Лістинг функції <назва першої функції>

```lisp
<Лістинг реалізації першої функції>
```

Тестові набори та утиліти

```lisp
<Лістинг реалізації утилітних тестових функцій та тестових наборів першої
функції>```

### Тестування
```

```

```lisp
<Виклик і результат виконання тестів першої функції>
```

## Лістинг функції <назва другої функції>

```lisp
<Лістинг реалізації другої функції>
```

### Тестові набори та утиліти

```lisp
<Лістинг реалізації утилітних тестових функцій та тестових наборів другої функції>
```

### Тестування

```lisp
<Виклик і результат виконання тестів другої функції>
```

```

На захисті лабораторної роботи необхідно продемонструвати роботу реалізованих функцій для різних тестових наборів, а також відповісти на питання викладача по виконаній роботі та теорії. Контрольні питання наведені в п 2.4.

2.3 Просте модульне тестування

Для виконання тестування розроблених функцій можна написати одну функцію, що виконує перевірку фактичного результату з очікуванням і виводить повідомлення про те, чи пройшла перевірка, чи ні. Наприклад:

```

;;; Test function `my-reverse'

(defun check-my-reverse (name input expected)
  "Execute `my-reverse' on `input', compare result with `expected' and print
  comparison status"
  (format t "~:[FAILED~;passed~]... ~a~%"
    (equal (my-reverse input) expected)
    name))

```

Функція `format` в залежності від значення першого аргументу форматowanego рядка (третій аргумент `format`) виводить "passed" або "FAILED", а також ім'я тесту, задане другим аргументом для форматowanego рядка.

Тепер можна написати тестові набори:

```

(defun test-my-reverse ()
  (check-my-reverse "test 1" '(1 2 3) '(3 2 1))
  (check-my-reverse "test 2" nil nil)
  (check-my-reverse "test 3" '(1 (2) (3 (4))) '((3 (4)) (2) 1)))

```

І виконати тести:

```
CL-USER> (test-my-reverse)
passed test 1
passed test 2
passed test 3
NIL
```

2.4 Контрольні питання

1. Що таке функціональне програмування?
2. Наведіть основні принципи функціонального програмування.
3. Що таке рекурсія?
4. Які є види рекурсії? Опишіть їх.
5. Як визначаються функції в Common Lisp?
6. Які є форми розгалуження виконання в Common Lisp?
7. Яка область видимості змінних в Common Lisp?
8. Що таке хвостова рекурсія? Які її особливості?

Лабораторна робота 3. Функціональний і імперативний підходи до роботи зі списками

Мета лабораторної роботи: навчитись застосовувати функціональний (рекурсія і конструктивний підхід) і імперативний (цикли і деструктивний підхід) підходи до роботи зі списками при реалізації програм мовою Common Lisp; навчитись застосовувати функціональний підхід при реалізації алгоритмів, що зазвичай реалізуються імперативно.

Конструктивний і деструктивний підходи розглянуті в розділі 2.3 навчального посібника. Конструктивні та деструктивні функції для роботи зі списками розглянуті в розділах 2.1, 2.3, 5.1 навчального посібника. Рекурсія розглянута в розділі 3, а циклічні конструкції — в розділі 2.6.3. Реалізація циклів за допомогою рекурсії розглянута в розділі 3.2.4. Функції для роботи з багатьма значеннями розглянуті в розділі 2.4.2.

Реченець виконання роботи (ака **дедлайн**) визначається викладачем під час видачі завдання на лабораторну роботу.

3.1 Завдання

Реалізуйте алгоритм сортування чисел у списку двома способами: функціонально і імперативно.

1. *Функціональний* варіант реалізації має базуватись на використанні рекурсії і конструюванні нових списків щоразу, коли необхідно виконати зміну вхідного списку. Не допускається використання: псевдо-функцій, деструктивних операцій, циклів. Також реалізована функція не має бути функціоналом (тобто приймати на вхід функції в якості аргументів).

2. *Імперативний* варіант реалізації має базуватись на використанні циклів і деструктивних функцій (псевдофункцій). Не допускається використання функцій вищого порядку або функцій для роботи зі списками/послідовностями, що використовуються як функції вищого порядку. Тим не менш, оригінальний список цей варіант реалізації також не має змінювати, тому перед виконанням деструктивних змін варто застосувати функцію `copy-list` (в разі необхідності). Також реалізована функція не має бути функціоналом (тобто приймати на вхід функції в якості аргументів).

Алгоритм, який необхідно реалізувати, задається варіантом (п. 3.1.1). Зміст і шаблон звіту наведені в п. 3.2.

Кожна реалізована функція має бути протестована для різних тестових наборів. Тести мають бути оформленні у вигляді модульних тестів (наприклад, як наведено у п. 2.3).

3.1.1 Варіанти алгоритмів сортування

Номер варіанту обирається як номер у списку групи, який надсилає викладач на початку семестру, *за модулем 5*: 1 -> 1, 2 -> 2, ..., 6 -> 1, 7 -> 2, ...

Варіант 1

Алгоритм сортування вибором за незменшенням.

Варіант 2

Алгоритм сортування обміном №1 (без оптимізацій) за незменшенням.

Варіант 3

Алгоритм сортування обміном №2 (із використанням прапорця) за незменшенням.

Варіант 4

Алгоритм сортування вставкою №1 (з лінійним пошуком зліва) за незменшенням.

Варіант 5

Алгоритм сортування обміном №4 ("шейкерне сортування") за незменшенням.

3.2 Формат звіту і захисту

В результаті виконання лабораторної роботи необхідно підготувати звіт у форматі Markdown. Файл звіту (з розширенням `.md`) та файл(и) з кодом (з розширенням `.lsp` або `.lisp`), або посилання на файл звіту і файл(и) з кодом у репозиторії (GitHub, Bitbucket, тощо.) необхідно надіслати викладачу на електронну пошту.

Звіт має містити (див. шаблон):

- титульні заголовки
- загальне завдання
- номер варіанту

- алгоритм сортування за варіантом
- лістинг реалізованих завдань
- лістинг реалізації тестових наборів та тестових утиліт
- результат виконання тестових наборів

Шаблон звіту такий:

```
<p align="center"><b>МОНУ НТУУ КПІ ім. Ігоря Сікорського ФПМ СПІСКС</b></p>

<p align="center">
<b>Звіт з лабораторної роботи 3</b><br/>
"Конструктивний і деструктивний підходи до роботи зі списками"<br/>
дисципліни "Вступ до функціонального програмування"
</p>

<p align="right"><b>Студент(-ка)</b>: Прізвище Ім'я По-батькові група</p>
<p align="right"><b>Рік</b>: рік</p>

## Загальне завдання

<!-- Зазначається загальне завдання -->

## Варіант <номер варіанту>

<!-- Зазначається алгоритм сортування за варіантом -->

## Лістинг функції з використанням конструктивного підходу

```lisp
<Лістинг реалізації з використанням конструктивного підходу>
```

### Тестові набори та утиліти

```lisp
<Лістинг реалізації утилітних тестових функцій та тестових наборів>
```

### Тестування

```lisp
<Виклик і результат виконання тестів>
```

## Лістинг функції з використанням деструктивного підходу

```lisp
<Лістинг реалізації з використанням деструктивного підходу>
```

### Тестові набори та утиліти

```lisp
<Лістинг реалізації утилітних тестових функцій та тестових наборів>
```
```

Тестування

```
```lisp
<Виклик і результат виконання тестів>
```
```

На захисті лабораторної роботи необхідно продемонструвати роботу реалізованих функцій для різних тестових наборів, а також відповісти на питання викладача по виконаній роботі та теорії. Контрольні питання наведені в п. 3.3.

3.3 Контрольні питання

1. Що таке псевдофункція? Наведіть приклади псевдофункцій.
2. В чому різниця між конструктивним та руйнівним підходами до роботи зі списками?
3. Наведіть приклади функцій для конструювання списків.
4. Наведіть приклади функцій, якими можна змінити списки деструктивно.
5. Опишіть особливості роботи макроса `SETF`.
6. Що таке "місця" (places), якщо говорити про деструктивні функції? Які є форми для роботи з "місцями"?
7. Як можна повернути з функції багато значень і обробити їх?
8. Як реалізувати цикли за допомогою рекурсії?

Лабораторна робота 4. Функції вищого порядку та замикання

Мета лабораторної роботи: навчитись реалізовувати і застосовувати функції вищого порядку та замикання при реалізації програм мовою Common Lisp.

Функції вищого порядку розглянуті в розділі 4 навчального посібника. Функції для роботи з послідовностями а також їхні типові ключові параметри розглянуті в розділі 5.1 навчального посібника. Замикання розглянуті в розділі 4.3 посібника.

Реченець виконання роботи (ака **дедлайн**) визначається викладачем під час видачі завдання на лабораторну роботу.

4.1 Завдання

Завдання складається з двох частин:

1. Переписати функціональну реалізацію алгоритму сортування з лабораторної роботи 3 з такими змінами:
 - о використати функції вищого порядку для роботи з послідовностями (де/якщо це доречно, в разі, якщо функції вищого порядку не були використані при реалізації л.р. №3);

- додати до інтерфейсу функції (та використання в реалізації) два ключових параметра: `key` та `test`, що працюють аналогічно до того, як працюють параметри з такими назвами в функціях, що працюють з послідовностями (р. 5.1). При цьому `key` має виконатись мінімальну кількість разів.

2. Реалізувати функцію, що створює замикання, яке працює згідно із завданням за варіантом (див. п 4.1.2). Використання псевдофункцій не забороняється, але, за можливості, має бути зменшене до необхідного мінімуму.

4.1.1 Варіанти завдань для частини першої

Варіанти завдань для першої частини завдання обираються ті ж, що і для третьої л.р.

4.1.2 Варіанти завдань для частини другої

Номер варіанту обирається як номер у списку групи, який надсилає викладач на початку семестру, *за модулем 12*: 1 -> 1, 2 -> 2, ..., 13 -> 1, 14 -> 2, ...

Варіант 1

Написати функцію `add-prev-fn`, яка має один ключовий параметр — функцію `transform`. `add-prev-fn` має повернути функцію, яка при застосуванні в якості першого аргументу `mapcar` разом з одним списком-аргументом робить наступне: кожен елемент списку перетворюється на точкову пару, де в комірці CAR знаходиться значення поточного елемента, а в комірці CDR знаходиться значення попереднього елемента списку. Якщо функція `transform` передана, тоді значення поточного і попереднього елементів, що потраплять у результат, мають бути змінені згідно `transform`. `transform` має виконатись мінімальну кількість разів.

```
CL-USER> (mapcar (add-prev-fn) '(1 2 3))
((1 . NIL) (2 . 1) (3 . 2))
CL-USER> (mapcar (add-prev-fn :transform #'1+) '(1 2 3))
((2 . NIL) (3 . 2) (4 . 3))
```

Варіант 2

Написати функцію `add-prev-reducer`, яка має один ключовий параметр — функцію `transform`. `add-prev-reducer` має повернути функцію, яка при застосуванні в якості першого аргументу `reduce` робить наступне: кожен елемент списку-аргументу `reduce` перетворюється на точкову пару, де в комірці CAR знаходиться значення поточного елемента, а в комірці CDR знаходиться значення попереднього елемента списку (тобто того, що знаходиться "зліва"). Якщо функція `transform` передана, тоді значення поточного і попереднього елементів, що потраплять у результат, мають бути змінені згідно `transform`. Обмеження, які накладаються на використання функції-результату `add-prev-reducer` при передачі у `reduce` визначаються розробником (тобто, наприклад, необхідно чітко визначити, якими мають бути значення ключових параметрів функції `reduce` `from-end` та `initial-value`). `transform` має виконатись мінімальну кількість разів.

```
CL-USER> (reduce (add-prev-reducer)
                  '(1 2 3))
```

```

      :from-end ...
      :initial-value ...)
((1 . NIL) (2 . 1) (3 . 2))
CL-USER> (reduce (add-prev-reducer :transform #'1+)
      '(1 2 3)
      :from-end ...
      :initial-value ...)
((2 . NIL) (3 . 2) (4 . 3))

```

Варіант 3

Написати функцію `add-next-fn`, яка має один ключовий параметр — функцію `transform`. `add-next-fn` має повернути функцію, яка при застосуванні в якості першого аргументу `mapcar` разом з одним списком-аргументом робить наступне: кожен елемент списку перетворюється на точкову пару, де в комірці CAR знаходиться значення поточного елемента, а в комірці CDR знаходиться значення наступного елемента списку. Якщо функція `transform` передана, тоді значення поточного і наступного елементів, що потраплять у результат, мають бути змінені згідно `transform`. `transform` має виконатись мінімальну кількість разів.

```

CL-USER> (mapcar (add-next-fn) '(1 2 3))
((1 . 2) (2 . 3) (3 . NIL))
CL-USER> (mapcar (add-next-fn :transform #'1+) '(1 2 3))
((2 . 3) (3 . 4) (4 . NIL))

```

Варіант 4

Написати функцію `add-next-reducer`, яка має один ключовий параметр — функцію `transform`. `add-next-reducer` має повернути функцію, яка при застосуванні в якості першого аргументу `reduce` робить наступне: кожен елемент списку-аргументу `reduce` перетворюється на точкову пару, де в комірці CAR знаходиться значення поточного елемента, а в комірці CDR знаходиться значення наступного елемента списку (тобто того, що знаходиться "справа"). Якщо функція `transform` передана, тоді значення поточного і наступного елементів, що потраплять у результат, мають бути змінені згідно `transform`. Обмеження, які накладаються на використання функції-результату `add-next-reducer` при передачі у `reduce` визначаються розробником (тобто, наприклад, необхідно чітко визначити, якими мають бути значення ключових параметрів функції `reduce` `from-end` та `initial-value`). `transform` має виконатись мінімальну кількість разів.

```

CL-USER> (reduce (add-next-reducer)
      '(1 2 3)
      :from-end ...
      :initial-value ...)
((1 . 2) (2 . 3) (3 . NIL))
CL-USER> (reduce (add-next-reducer :transform #'1+)
      '(1 2 3)
      :from-end ...
      :initial-value ...)
((2 . 3) (3 . 4) (4 . NIL))

```

Варіант 5

Написати функцію `propagator-fn`, яка має один ключовий параметр — функцію

`comparator` . `propagator-fn` має повернути функцію, яка при застосуванні в якості першого аргументу `mapcar` разом з одним списком-аргументом робить наступне: якщо елемент не "кращий" за попередній згідно з `comparator` , тоді він замінюється на значення попереднього, тобто "кращого", елемента. Якщо ж він "кращий" за попередній елемент згідно `comparator` , тоді заміна не відбувається. Функція `comparator` за замовчуванням має значення `#>` .

```
CL-USER> (mapcar (propagator-fn) '(1 2 3))
(1 2 3)
CL-USER> (mapcar (propagator-fn) '(3 1 4 2))
(3 3 4 4)
CL-USER> (mapcar (propagator-fn :comparator #'<) '(1 2 3))
(1 1 1)
```

Варіант 6

Написати функцію `rpropagation-reducer` , яка має один ключовий параметр — функцію `comparator` . `rpropagation-reducer` має повернути функцію, яка при застосуванні в якості першого аргумента `reduce` робить наступне: при обході списку з кінця, якщо елемент списку-аргумента `reduce` не "кращий" за попередній (той, що "справа") згідно з `comparator` , тоді він замінюється на значення попереднього, тобто "кращого", елемента. Якщо ж він "кращий" за попередній елемент згідно `comparator` , тоді заміна не відбувається. Функція `comparator` за замовчуванням має значення `#<` . Обмеження, які накладаються на використання функції-результату `rpropagation-reducer` при передачі у `reduce` визначаються розробником (тобто, наприклад, необхідно чітко визначити, якими мають бути значення ключових параметрів функції `reduce` `from-end` та `initial-value`).

```
CL-USER> (reduce (rpropagation-reducer)
                 '(3 2 1 2 3)
                 :from-end ...
                 :initial-value ...)
(1 1 1 2 3)
CL-USER> (reduce (rpropagation-reducer)
                 '(3 1 4 2)
                 :from-end ...
                 :initial-value ...)
(1 1 2 2)
CL-USER> (reduce (rpropagation-reducer :comparator #'>)
                 '(1 2 3)
                 :from-end ...
                 :initial-value ...)
(3 3 3)
```

Варіант 7

Написати функцію `remove-each-nth-fn` , яка має один основний параметр `n` та один ключовий параметр — функцію `key` . `remove-each-nth-fn` має повернути функцію, яка при застосуванні в якості першого аргументу `remove-if` робить наступне: кожен `n` -ний елемент списку-аргумента `remove-if` , для якого функція `key` повертає значення `t` (або не `nil`), видаляється зі списку. Якщо користувач не передав функцію `key` у `remove-each-nth-fn` , тоді зі списку видаляється просто кожен `n` -ний елемент.

```
CL-USER> (remove-if (remove-each-nth-fn 2) '(1 2 3 4 5))
(1 3 5)
CL-USER> (remove-if (remove-each-nth-fn 2 :key #'evenp) '(1 2 2 2 3 4 4 4 5))
(1 2 2 3 4 5)
```

Варіант 8

Написати функцію `remove-each-rnth-reducer`, яка має один основний параметр `n` та один ключовий параметр — функцію `key`. `remove-each-rnth-reducer` має повернути функцію, яка при застосуванні в якості першого аргумента `reduce` робить наступне: при обході списку з кінця функцією `reduce`, кожен `n`-ний елемент, з числа тих, для яких функція `key` повертає значення `t` (або не `nil`), видаляється зі списку. Якщо функція `key` повертає `nil`, тоді елемент пропускається. Якщо користувач не передав функцію `key` у `remove-each-rnth-reducer`, тоді зі списку видаляється просто кожен `n`-ний елемент. Обмеження, які накладаються на використання функції-результату `remove-each-rnth-reducer` при передачі у `reduce` визначаються розробником (тобто, наприклад, необхідно чітко визначити, якими мають бути значення ключових параметрів функції `reduce` `from-end` та `initial-value`).

```
CL-USER> (reduce (remove-each-rnth-reducer 2)
                  '(1 2 3 4 5)
                  :from-end ...
                  :initial-value ...)
(1 3 5)
CL-USER> (reduce (remove-each-rnth-reducer 2 :key #'evenp)
                  '(1 2 2 2 3 4 4 4 5)
                  :from-end ...
                  :initial-value ...)
(1 2 3 4 4 5)
```

Варіант 9

Написати функцію `duplicate-elements-fn`, яка має один основний параметр `n` та один ключовий параметр — функцію `duplicate-p`. `duplicate-elements-fn` має повернути функцію, яка при застосуванні в якості першого аргументу `mapcan` робить наступне: кожен елемент списку-аргумента `mapcan`, для якого функція `duplicate-p` повертає значення `t` (або не `nil`), дублюється `n` разів. Якщо користувач не передав функцію `duplicate-p` у `duplicate-elements-fn`, тоді дублюються всі елементи вхідного списку.

```
CL-USER> (mapcan (duplicate-elements-fn 2) '(1 2 3))
(1 1 2 2 3 3)
CL-USER> (mapcan (duplicate-elements-fn 2 :duplicate-p #'evenp) '(1 2 3 4 5))
(1 2 2 3 4 4 5)
```

Варіант 10

Написати функцію `duplicate-elements-reducer`, яка має один основний параметр `n` та один ключовий параметр — функцію `duplicate-p`. `duplicate-elements-reducer` має повернути функцію, яка при застосуванні в якості першого аргументу `reduce` робить наступне: при обході списку, кожен елемент списку-аргумента `reduce`, для якого функція `duplicate-p` повертає значення `t` (або не `nil`), дублюється `n` разів. Якщо користувач не передав функцію `duplicate-p` у `duplicate-elements-fn`, тоді

дублюються всі елементи вхідного списку. Обмеження, які накладаються на використання функції-результату `duplicate-elements-reducer` при передачі у `reduce` визначаються розробником (тобто, наприклад, необхідно чітко визначити, якими мають бути значення ключових параметрів функції `reduce` `from-end` та `initial-value`).

```
CL-USER> (reduce (duplicate-elements-reducer 2)
                  '(1 2 3)
                  :from-end ...
                  :initial-value ...)
(1 1 2 2 3 3)
CL-USER> (reduce (duplicate-elements-reducer 3 :duplicate-p #'evenp)
                  '(1 2 3)
                  :from-end ...
                  :initial-value ...)
(1 2 2 2 3)
```

Варіант 11

Написати функцію `merge-spinning-tuples-fn` , яка має один ключовий параметр — `shift-step` . `merge-spinning-tuples-fn` має повернути функцію, яка при застосуванні в якості першого аргументу `mapcar` робить наступне: об'єднує всі поточні елементи зі списків-аргументів `mapcar` в один список (кортеж), циклічно зміщуючи елементи в кортежі. Величина зміщення збільшується для кожного наступного кортежу на значення `shift-step` , починаючи з 0. Якщо `shift-step` не зазначений користувачем, тоді величина "кроку" зміщення — 1.

```
CL-USER> (mapcar (merge-spinning-tuples-fn) '(1 2 3) '(a b c))
((1 A) (B 2) (3 C))
CL-USER> (mapcar (merge-spinning-tuples-fn :shift-step 2)
                  '(a b c)
                  '(1 2 3)
                  '(d e f))
((A 1 D) (E B 2) (3 F C))
```

Варіант 12

Написати функцію `replacer` , яка має два основні параметри `what` і `to` та два ключові параметри — `test` та `count` . `replacer` має повернути функцію, яка при застосуванні в якості першого аргументу `reduce` робить наступне: при обході списку з кінця, кожен елемент списку-аргумента `reduce` , для якого функція `test` , викликана з цим елементом та значенням `what` , повертає значення `t` (або не `nil`), замінюється на значення `to` . Якщо `count` передане у функцію, заміна виконується `count` разів. Якщо `count` не передане тоді обмежень на кількість разів заміни немає. `test` має значення за замовчуванням `#'eq1` . Обмеження, які накладаються на використання функції-результату `replacer` при передачі у `reduce` визначаються розробником (тобто, наприклад, необхідно чітко визначити, якими мають бути значення ключових параметрів функції `reduce` `from-end` та `initial-value`).

```
CL-USER> (reduce (replacer 1 2)
                  '(1 1 1 4)
                  :from-end ...
                  :initial-value ...)
(2 2 2 4)
```

```
CL-USER> (reduce (replacer 1 2 :count 2)
                  '(1 1 1 4)
                  :from-end ...
                  :initial-value ...)
(1 2 2 4)
```

4.2 Формат звіту і захисту

В результаті виконання лабораторної роботи необхідно підготувати звіт у форматі Markdown. Файл звіту (з розширенням `.md`) та файл(и) з кодом (з розширенням `.lsp` або `.lisp`), або посилання на файл звіту і файл(и) з кодом у репозиторії (GitHub, Bitbucket, тощо.) необхідно надіслати викладачу на електронну пошту.

Звіт має містити (див. шаблон):

- титульні заголовки
- загальне завдання
- реалізацію першої частини завдання
 - номер варіанту першої частини
 - алгоритм сортування за варіантом
 - лістинг реалізованої програми
 - лістинг реалізації тестових наборів та тестових утиліт
 - результат виконання тестових наборів
- реалізацію другої частини завдання
 - номер варіанту другої частини
 - завдання за варіантом
 - лістинг реалізованої програми
 - лістинг реалізації тестових наборів та тестових утиліт
 - результат виконання тестових наборів

Шаблон звіту такий:

```
<p align="center"><b>МОНУ НТУУ КПІ ім. Ігоря Сікорського ФПМ СПІСКС</b></p>

<p align="center">
<b>Звіт з лабораторної роботи 4</b><br>
"Функції вищого порядку та замикання"<br>
дисципліни "Вступ до функціонального програмування"
</p>

<p align="right"><b>Студент(-ка)</b>: Прізвище Ім'я По-батькові група</p>
<p align="right"><b>Рік</b>: рік</p>
```

Загальне завдання

<!-- Зазначається загальне завдання -->

Варіант першої частини <номер варіанту>

<!-- Зазначається алгоритм сортування за варіантом -->

Лістинг реалізації першої частини завдання

```
```lisp
;;; Лістинг реалізації
```
```

Тестові набори та утиліти першої частини

```
```lisp
;;; Лістинг реалізації утилітних тестових функцій та тестових наборів
```
```

Тестування першої частини

```
```lisp
;;; Виклик і результат виконання тестів
```
```

Варіант другої частини <номер варіанту>

Лістинг реалізації другої частини завдання

```
```lisp
;;; Лістинг реалізації
```
```

Тестові набори та утиліти другої частини

```
```lisp
;;; Лістинг реалізації утилітних тестових функцій та тестових наборів
```
```

Тестування другої частини

```
```lisp
;;; Виклик і результат виконання тестів
```
```

На захисті лабораторної роботи необхідно продемонструвати роботу реалізованих функцій для різних тестових наборів, а також відповісти на питання викладача по виконаній роботі та теорії. Контрольні питання наведені в п. 4.3.

4.3 Контрольні питання

1. Що таке функція вищого порядку? Наведіть приклади.
2. Що таке замикання? Наведіть приклади.
3. Що таке послідовності? Які є функції для роботи з послідовностями?

4. Які основні ключові параметри у функцій для роботи з послідовностями?
5. Опишіть високорівневі предикати `EVERY` , `SOME` , `NOTANY` .
6. Опишіть допоміжні функції `IDENTITY` , `CONSTANTLY` , `COMPLEMENT` .
7. Опишіть функції для обходу і складання списків.

Додаток А. Поради щодо стилю написання коду мовою Common Lisp

Тут наведено деякі поради щодо оформлення коду мовою Common Lisp. Більш детальні описи конвенцій оформлення коду можна знайти за посиланнями:

- <https://lisp-lang.org/style-guide/>
- <https://google.github.io/styleguide/lispguide.xml>

А.1 Стиль написання коду

А.1.1 Пробіли між символьними виразами

При написанні програм в Common Lisp, слід дотримуватись наступних правил додавання пробілів і переносів рядка:

- після відкриваючої дужки перед першим елементом списку та після останнього елементу перед закриваючою дужкою пробіл не ставиться;
- між елементами списку ставиться по 1-му пробілу, або переносу рядка

Приклад:

```
(list (format nil "~A" arg) (1+ num) (- 10 k))
```

А.1.2 Відступи вкладених форм

Вкладені форми (або елементи списку), що записуються у новому рядку, мають мати певний відступ відносно батьківської форми (початку списку, його відкриваючої дужки). Мінімальний відступ — 1 пробіл, але типова кількість відступів залежить від конкретної форми. Наприклад, для `defun` , `let` та ін. "тіло" форми має відступ у два символи відносно початкового відступу батьківської форми.

Зазвичай кількість відступів регулюється Emacs-ом за допомогою комбінації C-M-q (Ctrl+Alt+q), застосованій на відкриваючій дужці форми (виправлення відбувається лише всередині форми, до якої ця комбінація застосована). Це правило стосується всіх вкладених форм.

Наприклад:

```
(defun foo (arg k)
  "Some comment about what this function does"
```

```
(if (eql arg k)
    (1+ arg)
    (case k
      (1 "ok")
      ((2 3)
       "better")
      (otherwise "bad"))))
```

А.1.3 Перенесення дужок, гілок та вкладених форм

Переносити дужки не потрібно. Коли реалізація деякої форми дописана і в кінці є кілька закриваючих дужок, вони мають бути розміщені одна за одною без переносів і пробілів:

```
(when a
  (when b
    (when c
      (+ a b c)))))
```

Гілки форм керування потоком виконання та вкладені форми слід переносити на наступний рядок, якщо всі вони не поміщаються в один.

Нема стандарту “максимальної довжини рядка”, й взагалі його довжина не є обмеженою. Але краще писати код так, щоб довжина його рядків не перевищувала 80-100 символів — так, щоб ширина рядка не перевищувала "половину екрана". Якщо ж форми доведеться переносити, то переносити потрібно всі, окрім однієї-двох перших. Якщо частину форм переносити, а частину — ні, то код може виглядати гірше або ж менш очевидно, бо деякі елементи "загубляться":

```
(if (eql (do-some-stuff (+ 2 3) (list 5)) (do-another-stuff (- 3 2))) (list 'good)
    (list 'bad))
```

У прикладі вище може здатись, що `(list 'bad)` — це Т-гілка оператора `if`. Проте насправді це NIL-гілка. А Т-гілка знаходиться рядком вище — `(list 'good)`. Тож варто Т-гілку також перенести на новий рядок:

```
;;; don't do this:
(if cond t-branch
    nil-branch)

;;; do this (when needed)
(if cond
    t-branch
    nil-branch)
```

`IF` може бути записаний в один рядок у випадку, якщо його загальна довжина невелика. Наприклад:

```
(if (eql x 1) 0 (error "X != 1"))
```

Якщо `IF` має дві гілки і перша перенесена на новий рядок рядок після умови, тоді й другу теж треба перенести. Тобто ось так писати погано:

```
(if cond
```

```
t-branch nil-branch) ; bad: nil-branch should be moved to new line
```

Крім того, не потрібно робити кілька перенесень. Достатньо одного. Тобто, наступний приклад є прикладом того, як робити не слід:

```
(if (eql arg k)
    ; empty line is bad
    (1+ arg)
    ; the same
    (case k
      (1 "ok")))
```

A.1.4 Іменування символів

Інтерпретатор дає змогу писати код в будь-якому регістрі — літери символів будуть приведені у верхній регістр автоматично (звісно, крім спеціального формату запису (`|a b|`)). Проте загальноприйнято писати програми в нижньому регістрі. Тому не варто писати `(Car ...)`, `(aPpEnd ...)`, і подібні варіанти з використанням змішаного регістру. Суто верхній регістр зазвичай не використовується при написанні коду.

Символи, що складаються з декількох слів записуються з дефісами між словами. Наприклад: `do-something-interesting`, `my-reverse`, `lab-1`. Це так званий стиль `kebab-case`.

При написанні програм на Lisp (як, загалом, і інших мовах програмування) слід уникати таких імен змінних, як `1` (маленька літера "l"), `0` (велика літера "o"), оскільки їх досить просто переплутати з одиницею `1` і нулем `0`.

A.1.5 Визначення функцій

Між окремими визначеннями функцій має бути відступ в один порожній рядок. Також можна їх розділяти за допомогою коментарів, наприклад, розділяючи групи функцій, що належать до різних завдань чи різних логічно пов'язаних груп. Наприклад:

```
(defun foo ()
  ...)

(defun bar ()
  ...)

;;; -----

(defun do-something (arg)
  ...)
```

При визначенні функції варто уникати занадто великих функцій або функцій, логіка яких неочевидна з коду. Але, якщо, все ж, функція виявилась складнішою, ніж варто, тоді варто додати документаційний коментар, в якому зазначено, що саме робить функція. Наприклад:

```
(defun reverse-list (lst)
  "Reverse input list" ; just as an example (since logic is obvious
                       ; from the function's name)
```

...)